



PROTOCOL DEVELOPER MANUAL

MyScoutee Registry Protocol

Version 1.0.0

A developer-facing specification and integration guide for the signed registry protocol, its append-only accounting and governance rails, privacy boundaries, verification rules, and compatibility contract.

TECHNOLOGY SENSITIVITY

The combined deployment topology, trust and key-handling rules, release and recovery controls, and protocol implementation details reduce the cost of reconnaissance for an unauthorized attacker. Distribute only through approved internal channels. Confidentiality supplements—never replaces—access control, signed artifacts, encryption, key protection, review, and executable conformance tests.

Document ID	MSC-RPDM-001
Document version	1.0.0
Release date	2026-07-30
Protocol version	1
Status	NORMATIVE V1.0.0

Document Control

Field	Value
Audience	Backend, client, security, integration, and test engineers
Purpose	Implement and verify interoperable Registry Protocol v1 clients
Normative language	<i>must</i> , <i>must not</i> , <i>required</i> , and <i>reject</i> define protocol requirements
Implementation basis	The Go implementation and repository reference specifications current on 2026-07-30
Status	Finished normative contract for Registry Protocol v1 as implemented by MyScoutee Registry 1.0.0
Classification	Confidential — Internal Distribution Only. Protocol details, Registry binaries, release artifacts, configuration, and key-handling information remain under approved internal access and distribution controls.
Operational companion	MyScoutee Registry Operator Manual, MSC-ROM-001

Confidentiality boundary. This finished protocol manual is internal implementation and integration material. It is not authorization to publish the Registry server, release artifacts, configuration, operational details, private keys, or governed records.

Contents

1. Scope and protocol model	7. Announcements and update manifests
2. Trust, identities, and canonical encoding	8. Cases, settlements, and exit rails
3. HTTP, signatures, idempotency, and errors	9. Client integration sequence
4. Registration and append-only ledger	10. Conformance and testing
5. Operator network protocol	11. Compatibility and security boundaries
6. Privacy-safe global identity	12. Appendix A: Endpoint matrix
	13. Appendix B: Selected canonical domains
	14. Appendix C: Evidence sources

1. Scope and Protocol Model

1.1 Purpose

Registry Protocol v1 lets independently operated MyScoutee deployments register a durable Ed25519 identity, submit signed aggregate facts, obtain registry-signed receipts, and participate in separately versioned operator, accounting, announcement, identity-deduplication, and exit-governance rails. It is deliberately not a remote administration protocol for a deployment.

The protocol implements four distinct assurance layers:

1. **Deployment proof:** a deployment proves possession of its private key.
2. **Registry receipt:** one immutable registry identity signs what it accepted.
3. **History integrity:** records are hash-linked and, where applicable, included in a compact Merkle tree.
4. **Governed interpretation:** operator review, eligibility, settlement, exit, and ownership actions use separate signed state machines.

1.2 Deliberate non-goals

Version 1 does not provide:

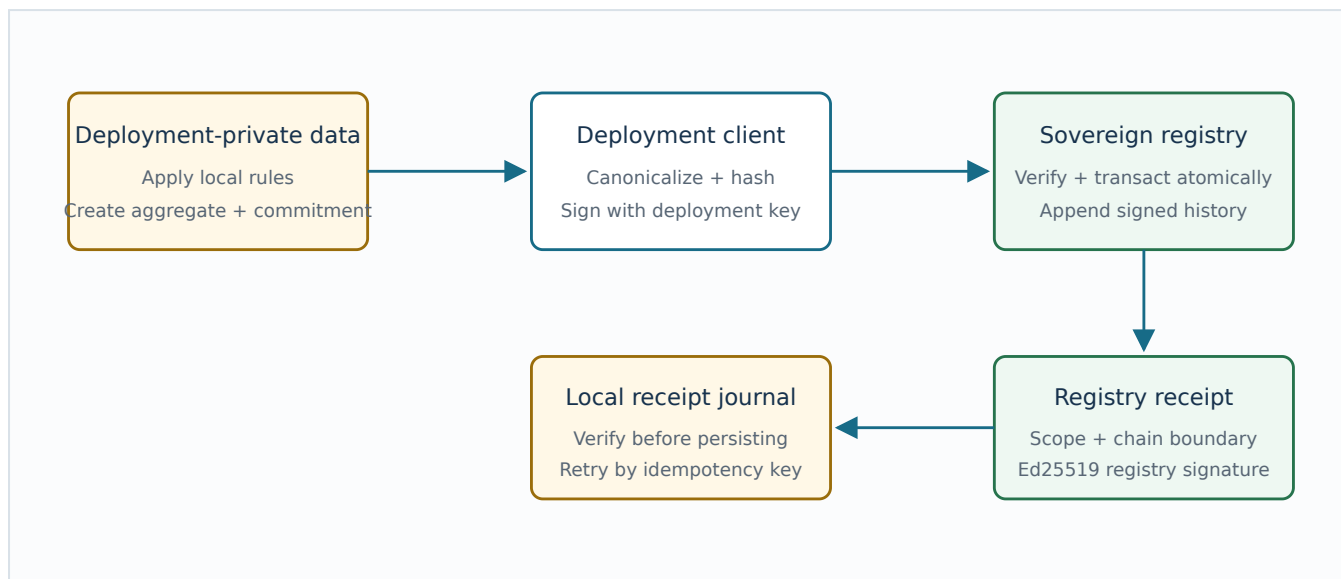
- registry-to-registry forwarding, federation, fallback, or replication;
- remote shell access, deployment configuration, artifact installation, or payment execution;
- raw user, payment, booking, activity, identity-document, or bank-account ingestion;
- proof that a deployment's private evidence is truthful merely because its aggregate is signed;
- anonymous private-set intersection or protection from a malicious VOPRF key holder;
- a legal entitlement, invoice, tax decision, purchase price, or automated payout.

1.3 Sovereign scope

Every registry has one mandatory and immutable `registry_scope`. It is 3-128 characters, starts with a lowercase ASCII letter or digit, and otherwise contains lowercase ASCII letters, digits, `.`, `:`, `_`, or `-`. A value such as `example:region-a` is a useful convention, not a reserved mode.

Scope is protocol domain separation: it is persisted with registry identity and is included in requests, receipts, ledger records, snapshots, and governance events. A deployment selects exactly one real registry endpoint, scope, and signing identity. A different scope is a different sovereign namespace even when both registries are operated by the same organization.

1.4 End-to-end protocol flow



The registry signs acceptance and historical boundaries. The deployment remains responsible for its private source evidence and for verifying every registry response before treating it as durable.

2. Trust, Identities, and Canonical Encoding

2.1 Cryptographic identities

Deployment and registry signatures use Ed25519. Public keys are encoded as complete X.509 SubjectPublicKeyInfo DER values and then standard padded RFC 4648 base64 in JSON.

```
public_key_fingerprint =  
  "sha256:" + lowercase_hex(SHA-256(complete SPKI DER))  
  
registry_key_id =  
  "rkey_" + first_32_hex_characters(fingerprint digest)  
  
package_signing_key_id =  
  "pkey_" + first_32_lowercase_hex_characters(SHA-256(complete SPKI DER))
```

Update packages use the independent deterministic `package_signing_key_id` shown above. A registry signing key must never be accepted implicitly as a package-signing key.

2.2 Build and Registry identity preflight

`GET /versionz` is a config-free, read-only build-information endpoint. A 1.0.0 server returns exactly:

```
{  
  "service": "myscoutee-registry",  
  "version": "1.0.0",  
  "protocol_version": "1"  
}
```

It is uncacheable operational metadata, not a signature or trust anchor. A client or release verifier must require the expected service, build version, and protocol version before continuing, then establish Registry identity through the signed endpoint below.

`GET /v1/registry/identity` returns protocol version, immutable scope, registry key ID, registry public key, and a self-signature over:

```
myscoutee-registry-identity-v1  
<protocol_version>  
<registry_scope>  
<registry_key_id>  
<registry_public_key>
```

A client must, in this order:

1. parse the SPKI value and require an Ed25519 public key;
2. derive and compare the deterministic `registry_key_id`;
3. reconstruct the canonical message and verify the self-signature;
4. display the exact endpoint, scope, and key ID for explicit operator approval;
5. pin all three values before registration.

Self-signature proves control of the advertised key, not that the endpoint is the intended business registry. TLS hostname verification and explicit operator trust confirmation remain required.

2.3 Wire encoding rules

Value	Canonical form
HTTP body	One UTF-8 JSON object; <code>Content-Type: application/json</code>
Protocol version	JSON string <code>"1"</code>
Timestamps	RFC 3339 UTC ending in literal <code>Z</code>
Month / date	<code>YYYY-MM / YYYY-MM-DD</code>
Binary	Standard padded RFC 4648 base64
Digest	<code>sha256:</code> plus 64 lowercase hexadecimal characters
Canonical integers	Base-10 with no leading <code>+</code> or leading zeros; a leading <code>-</code> appears only where the field permits a negative value
Canonical strings	Exact validated values; CR and LF are forbidden unless a field is explicitly hashed as raw UTF-8 content

2.4 Canonical message construction

JSON object order is never signed. Each protocol message defines an ordered line list beginning with a domain-separation string. Join fields with one LF byte and append exactly one final LF.

```
canonical(fields...) =
  UTF8(fields[0] + "\n" + fields[1] + "\n" + ... + fields[n] + "\n")
```

Do not sign serialized JSON, pretty-printed JSON, platform line endings, locale-formatted numbers, or a normalized value that differs from the transmitted validated value.

2.5 Hashes, links, and direct rows

The protocol uses SHA-256 for payload digests, immutable record hashes, prior-record links, evidence commitments, and Merkle nodes. The chain origin is:

```
sha256:0000000000000000000000000000000000000000000000000000000000000000
```

Append-only source records and their directly queryable rows are written in the same SQLite transaction. Direct rows are not asynchronous projections. Integrity verification derives the expected state and fails closed on mismatch; it does not silently rebuild or repair state while serving requests.

3. HTTP, Signatures, Idempotency, and Errors

3.1 Canonical request target

Signed endpoints cover the literal protocol path. Percent-encoded path aliases are rejected. Query strings are forbidden except on defined public cursor/filter reads such as announcements and leaderboards. Methods are exact and unsupported methods return `405` with an `Allow` header.

3.2 Generic signed request

```
myscoutee-registry-request-v1
<uppercase HTTP method>
<literal request path>
<protocol_version>
<registry_scope>
<signer>
<timestamp>
<nonce>
<idempotency_key>
<payload_hash>
```

Registration uses the submitted public-key fingerprint as `signer`. Post-registration operations use the assigned `deployment_id`. The default accepted timestamp skew is five minutes. Nonces, idempotency keys, and settlement query IDs use bytes `0x21 - 0x7e`: printable ASCII without spaces, 8–128 characters. For `POST /v1/settlements/query`, `query_id` occupies the generic canonical `<idempotency_key>` field position.

3.3 Mutation idempotency and replay behavior

The following rules apply to signed state-changing endpoints that append or update durable registry state:

1. Validate syntax, scope, signer, timestamp, payload hash, and signature before duplicate handling.
2. The same signer and idempotency key with the same semantic payload returns the original signed result with `duplicate: true`.
3. The same idempotency key with different content returns an idempotency conflict and appends nothing.
4. Reusing an accepted nonce for a different request returns a replay conflict and appends nothing.
5. A safe retry may use a fresh timestamp, nonce, and signature while preserving the original idempotency key and payload.

Clients must durably journal intent before sending and persist a verified receipt before acknowledging local completion. Network timeout is an unknown-result condition, not proof of failure.

`POST /v1/settlements/query` is a signed, repeatable read-only query, not a mutation-idempotent append. It uses `query_id` in the canonical idempotency-key position, but the registry does not persist a nonce or idempotency result for the query; a repeat may therefore carry a fresh response generation time and registry signature.

3.4 Strict JSON and body limits

The server accepts one JSON object, rejects invalid UTF-8, duplicate fields, unknown fields, trailing values, unsupported media type, and bodies over the configured limit. The default HTTP request-body limit is 64 KiB; identity-presence submission therefore uses bounded chunks.

3.5 Error envelope and status classes

```
{
  "error": {
    "code": "idempotency_conflict",
    "message": "bounded human-readable explanation"
  }
}
```

Status	Representative codes	Client action
400	<code>invalid_request</code> , <code>invalid_json</code> , <code>invalid_payload_hash</code> , <code>unsupported_protocol</code>	Fix request; do not retry unchanged
401	<code>invalid_signature</code>	Stop; inspect key, canonical message, and pin
404	<code>deployment_not_found</code> , <code>receipt_not_found</code> , <code>checkpoint_not_found</code>	Verify identifiers and lifecycle boundary
409	<code>idempotency_conflict</code> , <code>replay_conflict</code> , revision/state conflicts	Reconcile durable state; never invent a successor
413	<code>request_too_large</code>	Use the protocol's chunking or reduce bounded content
415	<code>unsupported_media_type</code>	Send <code>application/json</code>
429	<code>global_identity_rate_limited</code>	Back off; preserve idempotency intent
503	<code>registry_integrity_unavailable</code> , <code>global_identity_unavailable</code> , or clock unavailability	Do not bypass; retry only after registry recovery

Error messages are diagnostic text, not a stable machine contract. Branch on the bounded error code and HTTP status.

4. Registration and Append-only Ledger

4.1 Deployment registration

POST /v1/deployments/register proves possession of a submitted Ed25519 key. Its endpoint-specific payload is:

```
myscoutee-registry-registration-payload-v1
Ed25519
<base64 complete SPKI DER>
<canonical software version>
```

Hash the message, place the digest in `payload_hash`, then sign the generic request with the matching private key and its fingerprint as signer. Registration is also idempotent by public-key fingerprint: the same key cannot create two deployment identities.

The response assigns `dep_` plus 32 lowercase hexadecimal characters and includes a registry-signed registration receipt binding protocol, scope, deployment ID, public-key fingerprint, registration time, and registry key ID. A deployment ID is public routing identity, not an authentication secret; private-key possession controls the deployment.

4.2 Installation test

POST /v1/mau/batches accepts a non-accounting `installation-test` with ruleset `installation-test-v1` and count zero. It proves end-to-end signing, storage, receipt verification, and lookup without contributing to QMAU or leaderboard weight.

Use `period=YYYY-MM`, `revision=0`, and an empty `supersedes_batch_id`. Calculate `commitment_hash` as SHA-256 of:

```
myscoutee-registry-installation-test-v1
<deployment_id>
<idempotency_key>
```

As with every canonical message, each line above ends in LF, including the last one. Its `payload_hash` uses the installation-test batch domain `myscoutee-registry-mau-batch-payload-v1`, not the QMAU correction domain:

```
myscoutee-registry-mau-batch-payload-v1
installation-test
<YYYY-MM>
installation-test-v1
0
<commitment_hash>
```

4.3 Qualified monthly active users

Production snapshots use `kind=monthly-qmau` and `ruleset_version=qmau-v1`. One qualified user is a distinct local non-administrator, non-demo, non-test human account that records at least two substantive actions on at least two separate days in the applicable rolling 30-day activity window.

The deployment applies the rule to private evidence and sends only:

- the month, aggregate count, and opaque evidence commitment;
- a linear revision beginning at 1;
- the currently active prior batch ID for each correction.

Corrections are complete immutable replacements. Stale, skipped, or branching revisions fail. Historical revisions remain auditable; current reads select the newest valid revision at their captured boundary.

```
myscoutee-registry-qmau-batch-payload-v1
monthly-qmau
<YYYY-MM>
qmau-v1
<qualified_mau_count>
<commitment_hash>
<revision>
<supersedes_batch_id or empty>
```

The ordinary leaderboard measured weight is the arithmetic mean over the six most recent complete UTC months; a missing month contributes zero. This is deployment-level measured QMAU, not globally deduplicated humans.

4.4 Daily aggregate revenue

POST /v1/revenue/batches accepts per-currency aggregate totals under `daily-revenue`, `net-captured-revenue-v1`, and a fixed 500-basis-point commission rate. Its `period` is a valid UTC date in `YYYY-MM-DD` form. It accepts no user, customer, provider, payment, booking, or transaction identifiers.

Currency rows are strictly sorted and unique by ISO-4217 code. The implementation checks the expected fraction digits, keeps currencies separate, and limits a request to 32 currencies. The supported ISO-4217 code/exponent table is pinned by protocol v1. The `currencies` field must be a non-null array; `[]` records an explicit zero-revenue day. Each minor-unit value is at most 9,000,000,000,000, each `payment_count` is at most 1,000,000,000,000, and every row requires:

```
0 <= refunded_minor <= captured_minor
net_minor = captured_minor - refunded_minor
commission_basis_minor = net_minor
estimated_commission_minor =
  floor(commission_basis_minor * 500 / 10000)
payment_count >= 0
```

Revision and supersession are linear and immutable as for QMAU. Registry-wide commission pool calculations aggregate basis first and round once; they do not sum already-rounded deployment estimates.

4.5 Ledger and receipts

Accepted installation tests, QMAU, revenue, and settlement events append to one authoritative linear SHA-256 chain. A ledger entry commits to its one-based index, entry type, deployment or registry owner, batch identity, semantic metadata, payload hash, previous entry hash, and acceptance time. The write, source rows, direct query rows, Merkle update, and registry receipt are one SQLite transaction.

4.6 Compact Merkle proofs

The ledger also maintains an RFC 9162-style Merkle Tree Hash index without duplicating ledger leaves:

```
leaf_hash  = SHA-256(0x00 || raw_32_byte_ledger_entry_hash)
node_hash  = SHA-256(0x01 || raw_left_hash || raw_right_hash)
empty_root = SHA-256(empty bytes)
```

Inclusion and consistency proofs return an audit path and registry-signed tree head. Clients verify key identity, tree-head signature, and the complete path. These prove prefix inclusion or append-only growth; they do not prove the truthfulness of a deployment's private aggregate.

4.7 Completed-day checkpoints

GET /v1/ledger/checkpoints/{YYYY-MM-DD} returns an immutable registry-signed checkpoint after that UTC date closes. It commits to the through-ledger index, entry count, ledger head, previous checkpoint hash, generation time, scope, and key identity. Receipt lookup can identify the future checkpoint date before that date is finalized; callers must treat `checkpoint_not_finalized` as an expected lifecycle state.

5. Operator Network Protocol

5.1 Structured company claim

A registered deployment signs `POST /v1/operator/actions` with action `claim`. The v2 claim payload binds the legal name, registration number, jurisdiction, registered address, HTTPS website, verification contact, authority attestation, and optional HTTPS avatar. Surrounding whitespace and CR/LF are rejected.

Acceptance creates a `pending-review` generation and a provisional operator-group relationship. The public audit event includes a legal label and payload digest; private address and contact fields live in a separate append-only table in the same database. This is an access-control boundary, not field-level encryption.

5.2 Review and eligibility are separate rails

Review is a registry-local administrative action against one exact claim generation. Its status is `PENDING_REVIEW`, `APPROVED`, `REJECTED`, or `WITHDRAWN`. Eligibility is independently `active`, `suspended`, or `inactive`. These three values describe a direct claim/deployment. A leaderboard group aggregate can additionally report `partially-suspended` when it contains both active and suspended members.

Suspension preserves the approved claim, measured QMAU, and audit history while setting eligible weight and share to zero. Withdrawal or deployment deactivation makes the generation inactive. A later claim is a new generation and needs a new review.

5.3 Signed status

`GET /v1/operator/claims/{deployment_id}` binds the exact claim action, audit index/hash, group, legal name, review state, eligibility boundary, scope, and registry key. A client must verify this signature; browser presentation must not infer approval from group presence or a mutable label.

5.4 Client codes and grouping

The complete v1 operator action set is `claim`, `withdraw-claim`, `issue-client-token`, `revoke-client-token`, `redeem-client-token`, `revoke-group-link`, `deactivate-deployment`, and `reactivate-deployment`. The structured `claim` action uses its v2 claim payload; all other actions use the v1 operator action payload.

An approved deployment can issue a short-lived, single-use client code valid for 60–3600 seconds. Only its SHA-256 hash is stored and plaintext is returned only in the issue receipt. Redemption can link an already claimed deployment to the virtual group or copy the issuer's approved company submission into a new, independently reviewable pending claim.

Client codes never transfer deployment identity, private keys, QMAU, revenue, ledger ownership, or historical receipts. Deactivation withdraws the current claim and group link but preserves immutable history.

5.5 Snapshot-bound leaderboard

Public leaderboard pages are signed snapshots pinned to ledger, operator audit, review, eligibility, and ownership-transfer chain heads. Cursor continuation must retain the original snapshot and filters. A changed or tampered cursor fails closed.

The founder contribution is six months of 100,000 units. Founder share is no lower than one tenth; the remaining operator pool is allocated among eligible measured groups. Pending, rejected, withdrawn, inactive, and suspended generations may remain visible where specified but have zero eligible allocation weight.

6. Privacy-safe Global Identity

6.1 Protocol boundary

The optional extension recognizes the same verified person across deployments without sending the registry an email, telephone number, Firebase UID, provider subject, local profile ID, or plain identifier hash. It uses RFC 9497 VOPRF mode with suite `P256-SHA256`. Compressed P-256 elements are 33 bytes and DLEQ proofs are 64 bytes.

Normalization, input eligibility, consent capture, blinding, VOPRF finalization, and output derivation are deployment-local. The registry receives a blinded element for evaluation and later an opaque `sha256:` network identity commitment.

6.2 Key discovery and evaluation

`GET /v1/global-identities/voprf-keys/current` returns the active key version, suite, mode, compressed public key, activation time, and a registry signature. `POST /v1/global-identities/evaluate` is deployment-signed and returns evaluated element, proof, request hash, response hash, time, registry identity, and receipt signature.

The client must verify the registry's key-attestation signature, evaluate the DLEQ proof according to RFC 9497, finalize locally, and domain-separate the resulting commitment. Evaluations are idempotently audited and the implementation accepts at most 60 per deployment in a rolling minute.

6.3 Link, unlink, and correct

`POST /v1/global-identities/links` creates an explicit link with key version, suite, network commitment, consent version `global-dedup-consent-v1`, opaque consent-evidence commitment, verification timestamp, and effective month.

`POST /v1/global-identities/link-actions` accepts `UNLINK` or `CORRECT`. Correction supplies a replacement made under the currently active key and is the explicit rotation path. Unlink removes future applicability but does not rewrite historically valid period evidence.

6.4 Chunked qualified presence

Exact cross-deployment deduplication requires a presence submission that matches an accepted QMAU deployment, period, revision, superseded batch, and reported count. The deployment globally sorts commitments bitwise, retains duplicates, hashes the complete set, and divides it into independently signed ordered chunks under one stable submission ID.

The Go limit is 4096 commitments per chunk and 4096 chunks. The current Java sender uses 512 commitments per chunk to stay below the default HTTP body limit, producing an interoperable ceiling of 2,097,152 commitments per submission. Incomplete chunks are append-only staging and do not affect the public event chain or current dedup snapshot. The final missing chunk atomically completes the batch, event, and snapshot.

6.5 Aggregate result

```
deduplicated_network_qmau =  
  globally_unique_linked_count + unlinked_qmau_count  
  
duplicate_reduction =  
  linked_observation_count - globally_unique_linked_count
```

GET /v1/global-identities/dedup/{YYYY-MM} returns the latest immutable aggregate revision and its through-event boundary. It is a network-total measurement only. It does not redistribute one person's activity or weight, change deployment QMAU, or decide settlement.

6.6 Threat-model limit

V1 is privacy-enhancing, not anonymous PSI. A registry operator that can access both the VOPRF secret and restricted commitment tables can evaluate a dictionary of likely low-entropy identifiers and compare results.

Separate access to the VOPRF keyring and restricted tables, apply least privilege, audit use, encrypt storage, rate-limit evaluation, and obtain user consent. Do not claim full anonymity. Threshold OPRF or a separately governed/HSM-backed evaluator is a possible future protocol, not v1.

7. Announcements and Update Manifests

7.1 Signed public pull feed

GET `/v1/announcements` is a public, read-only, newest-first feed. A registry never opens an inbound deployment connection, pushes a command, downloads a package, or starts an upgrade. Filters include kind, severity, lowercase channel, expiry inclusion, limit, and a registry-signed cursor.

The first page freezes scope, registry key, one `as_of` time, sequence/head boundary, and filters. Each entry is signed and hash-linked; the snapshot and cursor are also signed. Clients must repeat the exact filters when following a cursor.

7.2 Content canonicalization

Kinds are `GENERAL`, `UPDATE`, `MAINTENANCE`, and `SECURITY`; severity is `INFO`, `NOTICE`, `WARNING`, or `CRITICAL`. Human content uses either translation keys or bounded locale objects. Locale and link sets are sorted before hashing. Title and body are hashed as raw UTF-8 bytes, allowing line breaks without violating line-oriented outer canonicalization. Links must be absolute HTTPS URLs without credentials or fragments.

7.3 Update manifest is an offer

Every `UPDATE` contains manifest version `1`, a canonical Semantic Version and channel, compatibility bounds, HTTPS `.deb` artifact and release-note URLs, artifact size and SHA-256, backup/downtime metadata, and state `AVAILABLE`, `REVOKED`, or `SUPERSEDED`.

```
myscoutee-release-package-v1
<release_version>
<channel>
<artifact_size_bytes>
<artifact_sha256>
```

The detached package signature is Ed25519 over that canonical message and is checked against a separately pinned package-signing public key. The registry validates and signs the manifest, but its signature does not authorize installation. Download, digest verification, package signature verification, backup, privileged installation, and operator consent remain local deployment responsibilities.

7.4 Publication boundary

There is no public mutation endpoint. A controlled local registry command accepts one strict reviewed JSON object, limits it to 256 KiB, and appends it immutably. Reusing `publication_id` with normalized identical content returns the original result; changed content conflicts. Revocation and supersession are new signed entries, never edits.

8. Cases, Settlements, and Exit Rails

8.1 Registry cases

The registry-local case rail records an anomaly observation against one existing deployment, claim, group, QMAU batch, revenue batch, or ledger index. `OPEN` and `CLEARED` do not alter claims, eligibility, measured accounting, leaderboard share, or legal rights. Events are globally hash-linked and registry-signed; current rows are maintained in the same transaction.

Records contain bounded category, severity, actor identifier, external reference, and optional evidence digest. Evidence bodies, names, email, credentials, and investigation notes stay in a separately governed system.

8.2 Monthly technical settlement

`share-weighted-settlement-v1` calculates a completed UTC month and one ISO-4217 currency. It pins ledger, operator audit, review, eligibility, selected revenue revisions, trailing-twelve-month basis, and exact beneficiary memberships. Recalculating an unchanged source returns the existing revision; a changed source appends a successor.

```
monthly_pool =  
  floor(monthly_commission_basis_minor * 500 / 10000)
```

Founder share is at least 10%; the eligible operator pool is divided by exact rational QMAU weights. Minor units use largest remainder, resolving equal remainders by ascending stable beneficiary ID. If no operator group is eligible, the operator pool goes to `UNALLOCATED_RESERVE`. Allocation conserves the pool exactly.

The separate `three-month-acceleration-valuation-v1` computes a bounded non-binding indicator from TTM commission-basis revenue. It is not a purchase price or payout. All exposed monetary integers remain in `0..9007199254740991` so JavaScript, Java, and Go represent them exactly.

`POST /v1/settlements/query` is a deployment-signed private history query. It returns only operator-group allocations for which the immutable historical source membership contains that deployment. Joining later does not reveal earlier allocation history.

8.3 Exit review

An exit review freezes one completed record date, checkpoint, ledger and Merkle prefix, exact claim generation, group membership and eligibility boundaries, and the latest settlement revision for each covered period/currency. Only one review can exist for an exact target claim generation.

Action	Allowed source	Result
<code>freeze</code>	No review	<code>review-pending</code>
<code>verify</code>	Pending or disputed	<code>verified-eligible</code>
<code>reject</code>	Pending or disputed	<code>rejected</code>

Action	Allowed source	Result
dispute	Verified or rejected	disputed
withdraw	Any non-withdrawn state	withdrawn , terminal

Buyer/auditor actor identifiers and evidence commitments are audit references, not human authentication. The registry signature proves what the registry recorded, not the legal identity of a person.

8.4 Ownership transfer

Ownership transfer is a registry-local manager state machine pinned to an exact verified exit, approved and eligible claim generation, source group, and target operator group. Preparation records intent; a registry-manager may approve or reject it, an open transfer may be cancelled, and completion is a deliberately separate terminal event. Completion is the only event that appends a dated, event-bound membership boundary for future effective membership. It never rewrites earlier membership, claims, snapshots, receipts, or settlements. Rejection or cancellation leaves immutable evidence and appends no membership boundary.

It transfers an operator-network relationship, not a deployment key, ledger row, historical receipt, company ownership, or payment instruction.

8.5 Final exit allocation

One final allocation may be created from a `verified-eligible` exit review using either an exact completed ownership transfer or an explicit no-transfer beneficiary. It copies the frozen settlement sources rather than selecting current settlements. Creation yields `recorded`; independent verification yields the terminal `verified-final`.

```
for every currency C:
  distributable_minor(C) =
    sum(frozen source-group network-pool allocations in C)

  allocated_minor(C) = distributable_minor(C)
```

Currencies are never combined or converted. Zero-valued currencies remain explicit. The record contains contractual and evidence commitments but no bank account, tax, invoice, or payment-provider field. It is an auditable allocation record, not payment execution.

9. Client Integration Sequence

9.1 Prerequisites

A conforming client needs:

- a trusted HTTPS implementation with hostname validation;
- Ed25519, SHA-256, X.509 SPKI, padded base64, strict UTF-8, and RFC 3339 UTC support;
- durable private-key storage and a restricted receipt/outbox journal;
- a reliable UTC clock within the registry's accepted skew;
- exact canonical-message builders shared with verification code;
- RFC 9497 P256-SHA256 support only when global identity is enabled.

9.2 Trust bootstrap

1. Receive a candidate HTTPS URL, expected release version, and expected scope from an authenticated operator workflow.
2. Fetch `/versionz` and require the expected service, release version, and protocol version without creating state.
3. Fetch Registry identity without creating state.
4. Verify TLS, Ed25519 SPKI, deterministic key ID, and registry self-signature.
5. Display the exact URL, scope, and key ID; require explicit confirmation.
6. Persist the pin and generate a deployment Ed25519 key in the protected host boundary.
7. Journal registration intent; sign and send registration.
8. Verify the returned scope, deployment ID, public-key fingerprint, registry key, and receipt signature before persisting.
9. Submit a zero-count installation test and verify its receipt and later lookup.

9.3 Durable mutation-submission algorithm

```
validate domain object locally
canonical = endpointSpecificCanonicalPayload(object)
payloadHash = sha256(canonical)

persist NEW outbox intent:
  endpoint path, scope, signer, idempotency key,
  semantic payload, payload hash

for each attempt:
  create fresh UTC timestamp and nonce
  sign generic request canonical bytes
  send one strict JSON object

on timeout:
  keep intent UNKNOWN and retry idempotently

on response:
  verify envelope, scope, identities, hashes,
```

```
chain boundaries and registry signature  
persist verified receipt and mark ACCEPTED
```

Never regenerate the idempotency key after a response-loss timeout. Never mark accepted from HTTP status alone. Never overwrite a prior receipt with a later query result unless the protocol defines a new immutable revision.

Apply this durable idempotency journal to state-changing submissions. For `POST /v1/settlements/query`, use the signed read-only query contract instead: `query_id` occupies the canonical idempotency-key position, but it is not a persisted mutation key and a repeated read can return a freshly generated signed response.

9.4 Response verification checklist

- Require the expected protocol version, registry scope, and pinned key ID/public key.
- Validate all identifiers, timestamps, digests, enums, ranges, and sorted collections.
- Rebuild canonical messages from the returned semantic fields.
- Recalculate payload, record, chain, content, allocation, or snapshot hashes as applicable.
- Verify Ed25519 signatures over canonical bytes, not response JSON.
- Check predecessor, revision, supersession, cursor, checkpoint, and Merkle boundaries.
- Reject partial verification; do not display unverified signed state as trusted.

9.5 Switching registry identity

Scope and registry key are not editable connection preferences. Moving a deployment to a different real registry requires an explicit governed transition, including signed deactivation at the old registry where applicable, a new local trust decision, and new registration semantics. Clients must not silently fail over, copy receipts between scopes, or accept a new key under the old pin.

10. Conformance and Testing

10.1 Canonical-vector tests

For every message builder, test exact bytes, one final LF, empty optional field lines, non-ASCII UTF-8, numeric boundaries, sorted collection behavior, and rejection of CR/LF. Include negative vectors for reordered lines, missing final LF, unpadded base64, uppercase digest hex, altered scope, and wrong domain separator.

10.2 Protocol behavior tests

- registration proof of possession and public-key idempotency;
- fresh-nonce retry of an already accepted idempotency key;
- idempotency-content conflict and nonce replay conflict;
- scope/key cross-registry rejection;
- QMAU and revenue correction linearity and stale-branch rejection;
- receipt lookup, completed-day checkpoint, Merkle inclusion, and consistency verification;
- operator claim/review/eligibility generation pinning;
- announcement cursor/filter binding and independent package-signature verification;
- VOPRF proof, link consent, key rotation correction, ordered chunk retry, and dedup snapshot verification;
- case, settlement, exit, transfer, and allocation state-machine invalid transitions.

10.3 Repository gates

```
go test ./...
go test -race ./...
go test -tags=e2e -count=1 -v ./e2e
go vet ./...
go build -trimpath -o /tmp/myscoutee-registry ./cmd/registry
docker build -t myscoutee-registry:qualification .
./tools/qualify-java-rails.sh
```

The tagged process suite exercises a real loopback registry with disposable SQLite and Ed25519 keys, including two simultaneous registries with distinct scopes and identities. The cross-language qualification runner exercises Java-generated signed QMAU, revenue, claim, and settlement requests against the Go server, response-loss recovery, process restart, correction revisions, and signature-tamper rejection.

10.4 What automated tests do not prove

Source tests do not authorize production rollout. Candidate qualification still requires exact-artifact, TLS, backup/restore, supported-upgrade, outage/retry, power-loss, load, multi-registry isolation, and independent security drills. Record artifact digests, source commit, commands, host, timings, and evidence for each release under qualification.

11. Compatibility and Security Boundaries

11.1 Version contract

`protocol_version` is the JSON string `"1"`. Domain-separation suffixes such as `-v1` and `-v2` identify exact canonical layouts; they do not all equal the top-level protocol version. For example, structured claims add source anchors through a v2 canonical operator receipt while preserving the original v1 layout for records where those anchors are absent.

A wire-compatible change may clarify validation, add a new endpoint, introduce a new explicitly selected ruleset, or add an optional JSON field only when old canonical messages and existing signed history retain their exact meaning. Changing line order, field interpretation, digest input, state transitions, or numeric encoding requires a new canonical domain and generally a protocol/ruleset version.

11.2 Fail-closed parsing

Requests reject unknown fields. Clients should also fail closed on unknown protocol versions, signature algorithms, mandatory enum values, canonical domains, or ruleset versions that affect accounting. An application may retain an unknown announcement kind only as untrusted opaque data; it must not infer update authorization from it.

11.3 Key boundaries

Key	Authority	Must not be used for
Deployment Ed25519	One deployment’s signed requests	Registry receipts, another deployment, package releases
Registry Ed25519	One immutable registry scope and its receipts/events	Package authorization or another scope
Package Ed25519	Detached release-package authenticity	Registry identity or deployment control
VOPRF keyring	Blind evaluation and commitment continuity	Ed25519 signatures or proof of user consent

SQLite database, registry signing key, registry identity, and VOPRF keyring form a coordinated recovery boundary. Replacing a missing key is not recovery. A restored database with a different key or scope must fail startup.

11.4 Data-minimization boundary

Keep raw evidence in its authoritative deployment, reviewer, buyer, or auditor system. Registry fields called `evidence_hash`, `commitment_hash`, `reference`, or `actor_id` are bounded commitments/identifiers, not places for documents, free-form notes, credentials, personal names,

or email addresses. Company verification contact data is a documented restricted exception and requires volume, CLI, log, and backup access controls.

11.5 Integrity over availability

Startup, health, writes, checkpoint finalization, and reads that require signed state verify the applicable integrity boundary. Clock rollback, key mismatch, missing source, chain corruption, invalid signature, or direct-row mismatch causes a fail-closed error. Clients must not bypass registry unavailability by accepting unsigned cached state or a different endpoint.

11.6 Semantic and legal boundaries

A valid signature proves that a key approved specific bytes. It does not prove a human identity, business authority, factual truth, legal ownership, contractual entitlement, tax result, or completed payment.

Those meanings arise only from the explicit review and governance rail appropriate to the record, plus the external legal and evidence systems referenced by bounded commitments.

Appendix A. Endpoint Matrix

Method and path	Authentication / result
GET /versionz	Config-free build and protocol version; read-only and not a trust bootstrap
GET /healthz	Operational health and integrity boundary; not a trust bootstrap
GET /v1/registry/identity	Public registry self-attestation; read-only
POST /v1/deployments/register	Submitted-key proof of possession; registration receipt
POST /v1/mau/batches	Deployment-signed installation test or QMAU snapshot
GET /v1/mau/batches/{batch_id}/receipt	Immutable MAU receipt lookup
POST /v1/revenue/batches	Deployment-signed daily aggregate revenue
GET /v1/revenue/batches/{revbatch_id}/receipt	Immutable revenue receipt lookup
GET /v1/ledger/checkpoints/{date}	Completed-UTC-day signed checkpoint
GET /v1/ledger/merkle/inclusion/{size}/{index}	Signed head and inclusion audit path
GET /v1/ledger/merkle/consistency/{old}/{new}	Signed head and append-only consistency path
POST /v1/operator/actions	Deployment-signed company/group/network action
GET /v1/operator/claims/{deployment_id}	Registry-signed direct claim and eligibility status
GET /v1/leaderboard	Public signed snapshot and cursor page
GET /v1/leaderboard/groups/{group_id}/deployments	Public snapshot-bound group members
GET /v1/global-identities/vopr-f-keys/current	Public registry-attested VOPRF key
POST /v1/global-identities/evaluate	Deployment-signed blinded evaluation

Method and path	Authentication / result
POST /v1/global-identities/links	Deployment-signed consent-bound link
POST /v1/global-identities/link-actions	Deployment-signed unlink or correction
POST /v1/global-identities/presence-batches	Deployment-signed ordered chunk
GET /v1/global-identities/dedup/{period}	Latest immutable network-dedup aggregate
GET /v1/announcements	Public signed filtered feed and cursor
POST /v1/settlements/query	Deployment-signed private historical allocation query

Case, manager review, eligibility, settlement calculation, exit review, ownership transfer, final allocation, key rotation, and announcement publication mutations are registry-local administrative interfaces, not public HTTP endpoints.

Appendix B. Selected Canonical Domains

This deliberately non-exhaustive index helps reviewers identify the main message families. It does not enumerate every payload, collection hash, receipt, event, snapshot, status, settlement-query, case, transfer, exit, or allocation domain. The implementation and the detailed protocol build inputs under [guides/manuals/protocols/](#) remain authoritative for exact domain strings and field order.

- myscoutee-registry-identity-v1
- myscoutee-registry-request-v1
- myscoutee-registry-registration-payload-v1
- myscoutee-registry-registration-receipt-v1
- myscoutee-registry-installation-test-v1
- myscoutee-registry-mau-batch-payload-v1
- myscoutee-registry-qmau-batch-payload-v1
- myscoutee-registry-ledger-entry-v1
- myscoutee-registry-mau-receipt-v1
- myscoutee-registry-qmau-receipt-v1
- myscoutee-registry-revenue-batch-payload-v1
- myscoutee-registry-revenue-receipt-v1
- myscoutee-registry-merkle-tree-head-v1
- myscoutee-registry-checkpoint-v1
- myscoutee-registry-operator-claim-payload-v2
- myscoutee-registry-operator-action-payload-v1
- myscoutee-registry-global-identity-vopr-key-v1
- myscoutee-registry-global-identity-evaluate-payload-v1
- myscoutee-registry-global-identity-link-payload-v1
- myscoutee-registry-global-identity-link-action-payload-v1
- myscoutee-registry-global-identity-presence-set-v1
- myscoutee-registry-global-identity-presence-chunk-v2
- myscoutee-registry-announcement-content-v1
- myscoutee-registry-announcement-links-v1
- myscoutee-registry-update-manifest-v1
- myscoutee-release-package-v1
- myscoutee-registry-case-event-v1
- myscoutee-registry-settlement-hash-v1
- myscoutee-registry-settlement-receipt-v1
- myscoutee-registry-exit-review-record-v1
- myscoutee-registry-ownership-transfer-record-v1
- myscoutee-registry-exit-allocation-record-v1

Appendix C. Evidence Sources

This manual is a cohesive developer guide. Exact field lists and canonical ordering should be checked against the current implementation and these repository evidence specifications during development and code review:

- `guides/manuals/protocols/protocol-v1.md`
- `guides/manuals/protocols/operator-network-v1.md`
- `guides/manuals/protocols/global-identity-v1.md`
- `guides/manuals/protocols/announcements-v1.md`
- `guides/manuals/protocols/registry-cases-v1.md`
- `guides/manuals/protocols/settlements-v1.md`
- `guides/manuals/protocols/exit-reviews-v1.md`
- `guides/manuals/protocols/ownership-transfers-v1.md`
- `guides/manuals/protocols/final-exit-allocations-v1.md`
- `guides/manuals/protocols/production-qualification.md`

Implementation packages reviewed

The corresponding Go evidence is under `internal/protocol`, `internal/httpapi`, `internal/service`, `internal/store/sqlite`, `internal/globalidentity`, and `cmd/registry`. Tests under those packages plus `e2e` define executable compatibility evidence.

MyScoutee Registry Protocol Developer Manual · MSC-RPDM-001 · Version 1.0.0 · 2026-07-30 · Confidential — Internal Distribution Only